# Coding Questions & Answers

4 Advanced Problems | Java & Python Solutions | Answer Key

## Problem 1: Trapping Rain Water

Given n non-negative integers representing an elevation map where the width of each bar is 1, compute how much water can be trapped after raining.

Water is trapped above a bar if there are taller bars on both its left and right. The amount of water above any position is determined by the shorter of the tallest bar to its left and the tallest bar to its right, minus the height at that position.

An O(n) time, O(1) space solution is expected. Brute-force O(n^2) approaches may time out on large inputs.

Input Format:
- A single line of space-separated non-negative integers representing the elevation map.

Output Format:
- Print a single integer: the total units of trapped rain water.

**SAMPLE INPUT:**

```
0 1 0 2 1 0 1 3 2 1 2 1
```

**SAMPLE OUTPUT:**

```
6
```

**CONSTRAINTS:**

1 <= n <= 2 * 10^4
0 <= height[i] <= 10^5
The answer can exceed the range of a 32-bit integer; use a 64-bit accumulator.

---

### Approach:

Use the two-pointer technique for O(n) time and O(1) space. Keep pointers at both ends with leftMax and rightMax. Always advance the side with the smaller current bar: the water trapped there is bounded by that side's running maximum (the opposite side is guaranteed to be taller). Add (max - height) at each step.

### Java Solution

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String[] parts = sc.nextLine().trim().split("\\s+");
        int n = parts.length;
        int[] height = new int[n];
        for (int i = 0; i < n; i++) height[i] = Integer.parseInt(parts[i]);
        int left = 0, right = n - 1, leftMax = 0, rightMax = 0;
        long water = 0;
        while (left < right) {
            if (height[left] < height[right]) {
                if (height[left] >= leftMax) leftMax = height[left];
                else water += leftMax - height[left];
                left++;
            } else {
                if (height[right] >= rightMax) rightMax = height[right];
                else water += rightMax - height[right];
                right--;
            }
        }
        System.out.println(water);
    }
}
```

## Python Solution

```
def solve():
    height = list(map(int, input().split()))
    n = len(height)
    left, right = 0, n - 1
    left_max = right_max = 0
    water = 0
    while left < right:
        if height[left] < height[right]:
            if height[left] >= left_max:
                left_max = height[left]
            else:
                water += left_max - height[left]
            left += 1
        else:
            if height[right] >= right_max:
                right_max = height[right]
            else:
                water += right_max - height[right]
            right -= 1
    print(water)
solve()
```

# Problem 2: Edit Distance

Given two strings word1 and word2, return the minimum number of single-character operations required to convert word1 into word2.

The permitted operations are:
- Insert a character
- Delete a character
- Replace a character

This is the classic Levenshtein distance problem. A dynamic-programming solution running in O(m * n) time is expected, where m and n are the lengths of the two strings.

Input Format:
- The first line contains word1.
- The second line contains word2.
(Either string may be empty, in which case the line is blank.)

Output Format:
- Print a single integer: the minimum edit distance between word1 and word2.

**SAMPLE INPUT:**

```
horse
ros
```

**SAMPLE OUTPUT:**

```
3
```

**CONSTRAINTS:**

0 <= word1.length, word2.length <= 500
Both strings consist of lowercase English letters.

---

## Approach:

Classic Levenshtein DP in O(m*n). Let dp[i][j] be the edit distance between the first i characters of word1 and the first j of word2. Base cases: dp[i][0] = i and dp[0][j] = j. If the characters match, dp[i][j] = dp[i-1][j-1]; otherwise it is 1 + min(replace = dp[i-1][j-1], delete = dp[i-1][j], insert = dp[i][j-1]).

## Java Solution

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String word1 = sc.hasNextLine() ? sc.nextLine() : "";
        String word2 = sc.hasNextLine() ? sc.nextLine() : "";
        int m = word1.length(), n = word2.length();
        int[][] dp = new int[m + 1][n + 1];
        for (int i = 0; i <= m; i++) dp[i][0] = i;
        for (int j = 0; j <= n; j++) dp[0][j] = j;
        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                if (word1.charAt(i - 1) == word2.charAt(j - 1)) {
                    dp[i][j] = dp[i - 1][j - 1];
                } else {
                    dp[i][j] = 1 + Math.min(dp[i - 1][j - 1],
                                   Math.min(dp[i - 1][j], dp[i][j - 1]));
                }
            }
        }
        System.out.println(dp[m][n]);
    }
}
```

## Python Solution

```
import sys
def solve():
    data = sys.stdin.read().split("\n")
    word1 = data[0] if len(data) > 0 else ""
    word2 = data[1] if len(data) > 1 else ""
    m, n = len(word1), len(word2)
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    for i in range(m + 1):
        dp[i][0] = i
    for j in range(n + 1):
        dp[0][j] = j
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if word1[i - 1] == word2[j - 1]:
                dp[i][j] = dp[i - 1][j - 1]
            else:
                dp[i][j] = 1 + min(dp[i - 1][j - 1], dp[i - 1][j], dp[i][j - 1])
    print(dp[m][n])
solve()
```

# Problem 3: Course Schedule

There are N courses labeled from 0 to N-1. Some courses have prerequisites: the pair "a b" means that to take course a you must first complete course b.

Determine whether it is possible to finish all N courses given the list of prerequisite pairs. It is possible if and only if the dependency graph contains no cycle.

A solution based on topological sorting (Kahn's algorithm using in-degrees, or DFS cycle detection) running in O(N + M) time is expected.

Input Format:
- The first line contains an integer N, the number of courses.
- The second line contains an integer M, the number of prerequisite pairs.
- The next M lines each contain two integers "a b", meaning course a depends on course b.

Output Format:
- Print "Yes" if all courses can be finished, otherwise print "No".

**SAMPLE INPUT:**

```
2
1
1 0
```

**SAMPLE OUTPUT:**

```
Yes
```

**CONSTRAINTS:**

1 <= N <= 10^5
0 <= M <= 10^5
0 <= a, b < N
There may be duplicate prerequisite pairs.

---

## Approach:

Model courses as a directed graph: 'a depends on b' becomes edge b -> a. All courses can be finished iff the graph is acyclic. Use Kahn's topological sort: push every node with in-degree 0, repeatedly pop a node and decrement its neighbours' in-degrees. If the number of processed nodes equals N, there is no cycle.

## Java Solution

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = Integer.parseInt(sc.nextLine().trim());
        int m = Integer.parseInt(sc.nextLine().trim());
        List<List<Integer>> adj = new ArrayList<>();
        for (int i = 0; i < n; i++) adj.add(new ArrayList<>());
        int[] indegree = new int[n];
        for (int i = 0; i < m; i++) {
            StringTokenizer st = new StringTokenizer(sc.nextLine());
            int a = Integer.parseInt(st.nextToken());
            int b = Integer.parseInt(st.nextToken());
            adj.get(b).add(a);
            indegree[a]++;
        }
        Deque<Integer> queue = new ArrayDeque<>();
        for (int i = 0; i < n; i++) if (indegree[i] == 0) queue.add(i);
        int processed = 0;
        while (!queue.isEmpty()) {
            int cur = queue.poll();
            processed++;
            for (int next : adj.get(cur)) {
                if (--indegree[next] == 0) queue.add(next);
            }
        }
```

```
        System.out.println(processed == n ? "Yes" : "No");
    }
}
```

## Python Solution

```
import sys
from collections import deque
def solve():
    data = sys.stdin.read().split("\n")
    idx = 0
    n = int(data[idx].strip()); idx += 1
    m = int(data[idx].strip()); idx += 1
    adj = [[] for _ in range(n)]
    indegree = [0] * n
    for _ in range(m):
        a, b = map(int, data[idx].split()); idx += 1
        adj[b].append(a)
        indegree[a] += 1
    queue = deque(i for i in range(n) if indegree[i] == 0)
    processed = 0
    while queue:
        cur = queue.popleft()
        processed += 1
        for nxt in adj[cur]:
            indegree[nxt] -= 1
            if indegree[nxt] == 0:
                queue.append(nxt)
    print("Yes" if processed == n else "No")
solve()
```

# Problem 4: Largest Rectangle in Histogram

Given an array of integers representing the heights of bars in a histogram where each bar has width 1, find the area of the largest rectangle that can be formed within the bounds of the histogram.

The rectangle must be axis-aligned and its height is limited by the shortest bar it spans. An O(n) solution using a monotonic stack is expected; the O(n^2) brute force may time out.

Input Format:
- A single line of space-separated non-negative integers representing bar heights.

Output Format:
- Print a single integer: the maximum rectangular area.

**SAMPLE INPUT:**

```
2 1 5 6 2 3
```

**SAMPLE OUTPUT:**

```
10
```

**CONSTRAINTS:**

1 <= n <= 10^5
0 <= height[i] <= 10^4
The answer can exceed the range of a 32-bit integer; use a 64-bit accumulator.

---

## Approach:

Use a monotonic increasing stack of bar indices in O(n). Iterate over the bars plus one sentinel of height 0 at the end. While the current bar is shorter than the bar at the stack top, pop it: the popped bar's height times the width (bounded by the current index and the new stack top) is a candidate area. Track the maximum.

## Java Solution

```java
import java.util.*;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String[] parts = sc.nextLine().trim().split("\\s+");
        int n = parts.length;
        int[] height = new int[n];
        for (int i = 0; i < n; i++) height[i] = Integer.parseInt(parts[i]);
        Deque<Integer> stack = new ArrayDeque<>();
        long maxArea = 0;
        for (int i = 0; i <= n; i++) {
            int cur = (i == n) ? 0 : height[i];
            while (!stack.isEmpty() && height[stack.peek()] >= cur) {
                int h = height[stack.pop()];
                int width = stack.isEmpty() ? i : i - stack.peek() - 1;
                maxArea = Math.max(maxArea, (long) h * width);
            }
            stack.push(i);
        }
        System.out.println(maxArea);
    }
}
```

## Python Solution

```python
def solve():
    height = list(map(int, input().split()))
    n = len(height)
    stack = []
    max_area = 0
    for i in range(n + 1):
        cur = 0 if i == n else height[i]
        while stack and height[stack[-1]] >= cur:
```

```
            h = height[stack.pop()]
            width = i if not stack else i - stack[-1] - 1
            if h * width > max_area:
                max_area = h * width
        stack.append(i)
    print(max_area)
solve()
```